

VERDICT — APPLICATION SECURITY AUDIT REPORT

Target: katrixbee/verdict | Stack: Next.js 15, TypeScript, SQLite/Postgres | Date: August 25, 2026

1. EXECUTIVE SUMMARY & POSTURE RATING

An exhaustive, non-destructive application security assessment was conducted across all endpoints, configuration files, and data pipelines of the **VERDICT** platform. The application is evaluated as **HIGH RISK** for production deployment due to missing perimeter controls and client-side privilege trust in citizen voting features.

SECURITY DOMAIN	STATUS	CVSS BASE	ASSESSMENT RESULT
Input Validation & SSRF	HIGH RISK	7.5	Flawed suffix match allows SSRF bypass in <code>/api/proxy-image</code>
Authentication & Identity	HIGH RISK	7.2	Unauthenticated citizen ratings; client-asserted DigiLocker trust
State Management & Memory	HIGH RISK	7.0	Unbounded in-memory heap storage susceptible to Out-Of-Memory DoS
Edge Hardening & Rate Limiting	MEDIUM RISK	6.5	No global rate-limiting middleware on public or expensive APIs
Security Headers & CSP	MEDIUM RISK	6.1	Missing Content-Security-Policy, HSTS, and X-Frame-Options
Information Disclosure	LOW RISK	4.3	Raw runtime exception string leakage in API 500 error responses

2. DISCOVERED VULNERABILITY DETAILS

SEC-01: SERVER-SIDE REQUEST FORGERY VIA SUFFIX MATCH BYPASS

CVSS 7.5 — HIGH

File: `src/app/api/proxy-image/route.ts:53`

Mechanics: Domain validation uses `allowedDomains.some(d => hostname.endsWith(d))`. An attacker can register `attackerwikipedia.org` or `maliciouseci.gov.in`. Because the hostname ends with the whitelisted string, the check passes. Furthermore, missing protocol checks and unmetered memory buffering allow large-payload memory exhaustion.

```
// Current Vulnerable Code:
const isAllowed = allowedDomains.some((domain) => hostname.endsWith(domain));

// Recommended Defensive Fix:
const isAllowed = allowedDomains.some(
  (domain) => hostname === domain || hostname.endsWith(`.${domain}`)
);
```

SEC-02: UNAUTHENTICATED STATE MUTATION & CLIENT CLAIM SPOOFING

CVSS 7.2 — HIGH

File: `src/app/api/ratings/route.ts:20-22`

Mechanics: `POST /api/ratings` accepts ratings without requiring session tokens. The Zod schema accepts client-submitted booleans `{ isLocalVoter: true, digilockerVerified: true }` without verifying digital signatures with the DigiLocker API. An automated botnet can skew politician trust ratings en masse.

SEC-03: IN-MEMORY HEAP STATE & MEMORY EXHAUSTION DOS

CVSS 7.0 — HIGH

File: `src/lib/supabase.ts:5`

Mechanics: Ratings are held in a global JavaScript object in memory (`inMemoryRatings`). Continuous submissions will crash Node.js via heap exhaustion. Data is entirely lost on container restart. A persistent database with Row Level Security (RLS) is required.

SEC-04: MISSING GLOBAL RATE LIMITING & ABUSE CONTROLS

CVSS 6.5 — MEDIUM

File: `src/middleware.ts` (Missing)

Mechanics: The API perimeter has no token-bucket or sliding-window rate limiters. Search endpoints, image proxies, and profile queries can be scraped or flooded without friction.

SEC-05: MISSING ENTERPRISE HTTP DEFENSIVE HEADERS

CVSS 6.1 — MEDIUM

File: `next.config.js`

Mechanics: Response headers lack `Content-Security-Policy`, `Strict-Transport-Security`, `X-Frame-Options: DENY`, and `X-Content-Type-Options: nosniff`, leaving the site susceptible to clickjacking, MIME sniffing, and unauthorized asset execution.

SEC-06: INTERNAL EXCEPTION STRING DISCLOSURE IN API 500S

CVSS 4.3 — LOW

File: `src/app/api/politicians/route.ts:65`

Mechanics: Catch blocks serialize raw error objects directly to clients via `String(error)`, potentially disclosing file paths, database schemas, and stack traces.

3. PRIORITIZED REMEDIATION ROADMAP

PHASE	ACTION ITEM	TARGET FILES	EFFORT
Phase 1 (Immediate)	Patch SSRF (exact domain match + HTTPS enforcement + 5MB cap)	<code>src/app/api/proxy-image/route.ts</code>	1 Day
Phase 1 (Immediate)	Inject Security Headers (CSP, HSTS, X-Frame-Options)	<code>next.config.js</code>	1 Day
Phase 1 (Immediate)	Sanitize API error responses to opaque error objects	<code>src/app/api/*</code>	0.5 Day
Phase 2 (Identity)	Implement Server-Side Authentication & Session Cookies	<code>src/lib/supabase.ts</code>	3 Days

PHASE	ACTION ITEM	TARGET FILES	EFFORT
Phase 2 (Identity)	Verify DigiLocker Signatures on Backend (Server-to-Server)	src/app/api/ratings/route.ts	2 Days
Phase 2 (Database)	Deploy PostgreSQL Schema with Strict Row Level Security (RLS)	supabase/migrations/	2 Days
Phase 3 (Edge)	Deploy Edge Rate Limiting Middleware (Upstash / Redis)	src/middleware.ts	2 Days